

Formal Languages and Compilers

Prof. Breveglieri, Morzenti, Agosta

Written exam: laboratory question

13/02/2024

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

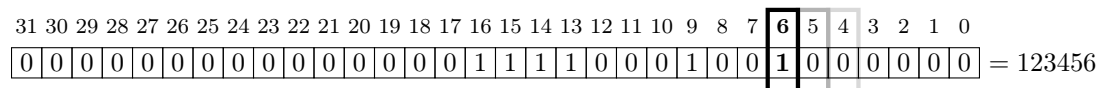
The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the Lance language with the ability to support the **pick** expression operator. This operator takes an array identifier and an expression as arguments:

pick ($\langle array \rangle$, $\langle exp \rangle$)

When it is computed, `pick` searches for the first bit set to 1 in the value of $\langle exp \rangle$, scanning the bits starting from the least significant one. Then it reads the element in $\langle array \rangle$ at the same *index* as the bit found, and it returns the value of the array element as the value of the operator. Bit indices are considered starting from 0 for the least significant bit.

As an example, consider the number 123456. The representation of this number in binary and the numbering of the bits is shown in the following figure.



When scanning the bits from right to left (i.e. from the least significant bit to the most significant) the first 1 we find is on bit 6. As a result, `pick` will read the array element at index 6 and return it.

If there are no bits set to 1 in the value of $\langle exp \rangle$, or if the index of the bit exceeds the bounds of $\langle array \rangle$, the value of the `pick` operator will be 0. Additionally, the modified compiler shall produce a syntax error if $\langle array \rangle$ is not a valid array identifier. No constant folding is required.

The following code snippet exemplifies the functionality of the operator.

```
int a[10], c;
// ... (array initialization omitted) ...
// a = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

c = pick(a, 123456) * 1024;    // The least significant bit set to 1 is bit 6
write(c);                     // Prints a[6] * 1024 = 7168
c = pick(a, c);                // The first bit set to 1 is bit 10, out of bounds!
write(c);                      // Prints 0
write(pick(a, c - 7168));      // The expression is equal to zero, prints 0
```

Tip: The following expression is true if bit n of variable x is set to 1: $(x \gg n) \ \& \ 1$

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (2 points)
2. Define the syntactic rules or the modifications required to the existing ones. (3 points)
3. Define the semantic actions needed to implement the required functionality. (20 points)

4. Given the following Lance code snippet:

```
(a & 12345 == 1024 & b) --5
```

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the exp nonterminal*. (5 points)

5. **(Bonus)** Briefly explain in plain English words how to modify the solution given to the questions about the `pick` operator in order to also reset the bit found equal to 1 to 0.

Notice that this extension requires modifying the syntax of `pick` to take a *scalar variable identifier* instead of an *expression* as second argument. Why?